

## 1. Syfte

Syftet med projektet var att fördjupa mina kunskaper inom datavalidering i Python med hjälp av biblioteket Pandera. Jag ville framför allt förstå hur man kan kontrollera kvaliteten på data innan den används i en analys och hur valideringsregler kan samlas på ett tydligt och återanvändbart sätt.

Som praktiskt exempel använde jag orderdata i CSV-format. Jag byggde ett mindre Pythonflöde där data först läses in med Pandas och sedan valideras med Pandera. Om datan uppfyller reglerna går den vidare till analys. Om datan innehåller fel stoppas analysen och valideringsfelen visas.

Jag valde också att jämföra Pandera med manuell validering i Pandas. Anledningen var att jag ville förstå vad Pandera faktiskt tillför, eftersom det även går att kontrollera data direkt med Pandas.

Projektet är avgränsat till orderdata från CSV-filer och ett mindre antal valideringsregler. Målet var inte att bygga ett komplett produktionssystem, utan att förstå hur ett strukturerat valideringssteg kan användas i ett Pythonbaserat dataflöde.

## 2. Området och dess relevans

Datavalidering innebär att kontrollera att data uppfyller vissa regler innan den används vidare. Detta är viktigt inom Data Science eftersom kvaliteten på indata påverkar analyser och de resultat som kommer efteråt.

I mitt projekt består orderdatan bland annat av order-ID, datum, kund-ID, produktkategori, antal, pris, rabatt och information om ordern är returnerad eller inte. Det räcker därför inte att kolumnerna finns. Värdena måste också vara rimliga och följa de regler som gäller för datan.

Exempelvis ska ett pris inte vara negativt. En rabatt på 1,25 är inte giltig i mitt projekt eftersom det motsvarar 125 procent. Ett värde som `not-a-date` kan inte användas som ett giltigt orderdatum. Även värdet `two` är felaktigt när kolumnen ska innehålla antal produkter.

Jag placerade därför valideringen mellan inläsning och analys:

**Data → Validering → Analys → Resultat**

På detta sätt kan problem upptäckas innan felaktiga värden används i beräkningar.

Pandas används i projektet för att läsa CSV-filer och arbeta med DataFrames. Pandera används för att beskriva hur DataFrame-datan ska se ut och vilka regler den måste följa. För mig blev det en tydlig skillnad mellan att hantera data och att definiera regler för datakvalitet.

## 3. Viktiga begrepp

### DataFrameSchema

Det centrala begreppet i projektet är `DataFrameSchema`. Ett schema beskriver vilka kolumner som ska finnas och vilka regler som gäller för datan.

I min lösning skapar funktionen `create_orders_schema()` ett schema för orderdatan. Där definieras bland annat datatyper, obligatoriska värden och kontroller.

Fördelen med detta är att reglerna samlas på ett ställe. Det blir därför lättare att se vilka antaganden programmet gör om datan.

### Column och Check

`Column` används för regler som gäller för en viss kolumn. `Check` används för att kontrollera ett villkor.

Ett exempel från min lösning är:

```
"discount": pa.Column(
    float,
    nullable=False,
    checks=pa.Check.between(0, 1)
)
```

Här finns tre delar. Kolumnen ska vara `float`, den får inte innehålla saknade värden och värdet ska ligga mellan 0 och 1.

Det är alltså inte bara en kontroll av datatypen. Schemat beskriver även vad som räknas som ett giltigt värde i projektet.

### **lazy=True**

Jag använder:

```
schema.validate(orders, lazy=True)
```

`lazy=True` gör att Pandera samlar flera valideringsfel i samma körning istället för att stoppa vid det första felet. Det är användbart när man vill få en bättre bild av datakvaliteten.

I mitt projekt innehåller den ogiltiga filen flera olika fel samtidigt. Därför blir det mer praktiskt att få flera fel rapporterade på en gång.

## **4. Genomförande**

Jag skapade tre typer av testdata. `valid_orders.csv` innehåller giltiga orderrader. `invalid_orders.csv` innehåller avsiktliga fel och `missing_column_orders.csv` saknar den obligatoriska kolumnen `discount`.

Den giltiga filen innehåller 20 orderrader. I den ogiltiga filen testade jag bland annat:

- `quantity = -1`
- `quantity = "two"`
- `unit_price = -349.50`
- `discount = 1.25`
- `order_date = "not-a-date"`
- `product_category = "Toys"`
- saknat `customer_id`
- felaktigt värde i `returned`

Jag använde Pandas för att läsa in CSV-filerna och skicka DataFrame-datan till Pandera för validering.

En teknisk del som krävde extra arbete var kolumnen `quantity`. I testdatan finns textvärdet `"two"`, trots att kolumnen ska representera antal. Jag valde därför att först behandla `quantity` som text:

```
orders["quantity"] = orders["quantity"].astype("string")
```

Sedan kontrollerar Pandera både om värdet kan konverteras till ett numeriskt värde och om det är större än noll.

Detta var ett medvetet tekniskt val. Jag ville att `"two"` skulle kunna upptäckas som ett valideringsfel istället för att problemet skulle döljas eller hanteras tidigare i flödet. Efter en lyckad validering konverteras kolumnen tillbaka till numeriskt format i analysdelen:

```
orders["quantity"] = pd.to_numeric(
    orders["quantity"]
)
```

Det visar också att validering och analys har olika uppgifter. Under valideringen behöver jag kunna identifiera felaktiga värden. När datan är godkänd behöver analysen däremot numeriska värden för sina beräkningar.

Jag valde även att dela upp programmet i olika delar. `validate_orders.py` ansvarar för valideringen, `analysis.py` för beräkningarna och `main.py` kopplar ihop flödet. På så sätt körs analysen endast när valideringen har lyckats.

## 5. Resultat

När jag körde programmet med den giltiga filen passerade datan valideringen. Analysen gav:

- **Antal order:** 20
- **Total försäljning:** 11 960,47
- **Genomsnittligt ordervärde:** 598,02

Programmet kunde även beräkna försäljningen per produktkategori.

När jag körde samma program med `invalid_orders.csv` misslyckades valideringen. Pandera hittade bland annat ogiltigt datum, saknat kund-ID, en otillåten produktkategori, ogiltiga kvantiteter, negativt pris och felaktig rabatt.

Det viktiga resultatet var inte bara att felen hittades. Analysen kördes inte när datan var ogiltig.

Flödet blev därför:

**Giltig data → validering lyckas → analys genomförs**

eller:

**Ogiltig data → validering misslyckas → felrapport → analys stoppas**

Jag testade även en fil där `discount` saknades helt. Pandera upptäckte då att en obligatorisk kolumn saknades. Detta visade att valideringen inte bara kan kontrollera värden i befintliga kolumner, utan även datans struktur.

### Pandera jämfört med manuell validering i Pandas

Jag gjorde även en enklare manuell validering med Pandas. Ett exempel är att kontrollera om rabatt saknas eller ligger över det tillåtna intervallet:

```
invalid = (
    orders["discount"].isna()
    | orders["discount"].gt(1)
)
```

Denna metod fungerar och ger stor flexibilitet. Om det bara finns några få och mycket specifika kontroller kan den vara enkel att förstå och använda.

Skillnaden i mitt projekt är att Pandera samlar reglerna i ett schema. Med Pandas behöver kontrollerna skrivas som separata villkor, medan Pandera gör det möjligt att samla reglerna i en gemensam beskrivning av den förväntade datan.

Jag ser därför inte Pandera som en ersättning för Pandas. Pandas är fortfarande viktigt för datahantering och analys. Pandera kan däremot ge mer struktur när valideringsreglerna blir fler, återkommer eller behöver vara tydligt dokumenterade.

## 6. Begränsningar och möjliga förbättringar

Projektet är en mindre prototyp och har därför flera begränsningar.

Den första begränsningen är att jag endast använder CSV som datakälla. I ett större dataflöde kan data komma från exempelvis en databas eller ett API. Projektet visar därför bara en liten del av hur Pandera kan användas.

Jag har också ett begränsat antal valideringsregler. Reglerna är anpassade till min orderdata. I en verklig verksamhet kan det finnas fler affärsregler och regler som beror på relationen mellan flera kolumner.

En möjlig förbättring är därför att lägga till fler kontroller på DataFrame-nivå. Det skulle exempelvis göra det möjligt att kontrollera regler som inte bara gäller en kolumn utan relationen mellan flera kolumner.

Jag skulle också kunna lägga till fler tester och mer realistiska datakällor. Ett annat möjligt steg är att integrera valideringen i ett större ETL-flöde och spara valideringsresultaten på ett mer standardiserat sätt för loggning och uppföljning.

Samtidigt tycker jag att projektet visar att mer komplexitet inte automatiskt betyder en bättre lösning. Om ett projekt bara behöver några få enkla kontroller kan manuell validering med Pandas fortfarande vara ett rimligt alternativ. Valet av teknik bör därför bero på projektets behov och hur många regler som ska underhållas.

## 7. Koppling till yrkesrollen

Datavalidering är relevant för Data Scientist och Data Analyst eftersom analys ofta bygger på data från olika källor. Data behöver därför kontrolleras innan den används vidare.

I mitt projekt blir Pandera ett separat steg mellan rådata och analys. Det liknar ett dataflöde där data först kontrolleras och sedan skickas vidare till nästa steg.

En viktig sak jag tar med mig är att datavalidering inte betyder att all data måste vara perfekt. Det viktiga är att bestämma vilka regler som måste vara uppfyllda för att nästa steg ska kunna använda datan på ett tillförlitligt sätt.

Projektet har också gett mig en bättre förståelse för varför tekniska verktyg behöver väljas utifrån problemet. Pandera passar bra när valideringsregler behöver vara strukturerade och återanvändbara, medan Pandas kan räcka för mindre och mer specifika kontroller.

## 8. Källor

- Pandera Documentation – DataFrame Schemas.
- Pandera Documentation – DataFrameSchema API Reference.
- Pandera Documentation – Lazy Validation.

- [Pandera Documentation – Check API Reference](#).
- [Pandas Documentation 3.0.6](#).

Källorna användes framför allt för att förstå `DataFrameSchema`, `Column`, `Check`, `validate()` och `lazy=True`, samt hur Pandas används för DataFrames och dataanalys.

## 9. Självreflektion

Det viktigaste jag lärde mig var hur datavalidering kan bli ett eget steg i ett Pythonbaserat dataflöde. Jag hade arbetat med Pandas tidigare, men genom projektet fick jag en bättre förståelse för hur ett schema kan användas för att beskriva vilka regler datan måste följa.

Jag lärde mig också hur `DataFrameSchema`, `Column` och `Check` fungerar tillsammans och varför `lazy=True` kan vara användbart när flera fel behöver hittas samtidigt.

Det som var svårast för mig var att förstå hur valideringsreglerna skulle byggas när testdatan innehöll olika typer av fel. Det handlade inte bara om felaktiga siffror, utan också om text där jag förväntade mig numeriska värden, saknade värden och felaktiga kategorier. Jag behövde därför förstå skillnaden mellan datans format och själva regeln som jag ville kontrollera.

Det tekniska val jag är mest nöjd med är hanteringen av `quantity`. Jag valde att tillfälligt behandla kolumnen som text före valideringen eftersom testdatan innehåller `"two"`. Då kunde jag kontrollera både om värdet var numeriskt och om det var större än noll. Efter valideringen konverteras värdet till numeriskt format innan analysen.

Om jag skulle göra projektet igen skulle jag planera valideringsreglerna mer systematiskt från början. Jag skulle också testa fler gränssfall, till exempel exakt 0 och 1 för rabatt och fler typer av saknade eller felaktiga värden.

Ett naturligt nästa steg skulle vara att använda valideringen i ett större dataflöde med exempelvis en databas eller ett API. Jag skulle också vilja undersöka fler kontroller som beror på flera kolumner samtidigt.

Jag bedömer att projektet motsvarar **VG** eftersom jag inte bara har fått Pandera att fungera, utan också undersökt hur och varför tekniken används. Jag har arbetat med centrala begrepp som `DataFrameSchema`, `Column`, `Check` och `lazy=True`. Jag har gjort ett eget tekniskt val kring `quantity`, jämfört Pandera med manuell validering i Pandas och diskuterat när de olika alternativen kan passa.

Samtidigt är projektet medvetet litet och inte en komplett produktionslösning. Min bedömning bygger därför främst på min förståelse av tekniken, mina tekniska val, fungerande exempel och min förmåga att förklara både möjligheter och begränsningar.